BASTION ARENA

Zero-Trust Infrastructure for Adversarial AI Safety Testing

Complete system design, implementation playbook, threat model, roadmap and market case
for Hackathon Problem 04 — *Securing Adversarial AI Safety Testing Infrastructure (Bayora)*

Tagline for the pitch: *"A red-team finding is only evidence if the lab that produced it can be trusted. Bastion Arena makes the lab provable."*

1. Winning thesis in one page

Most teams will answer this problem with "put each team in a container and add a firewall." That is necessary but not sufficient, and a good judge will see through it. The problem statement hides four contradictions. **Winning means resolving them explicitly** instead of ignoring them:

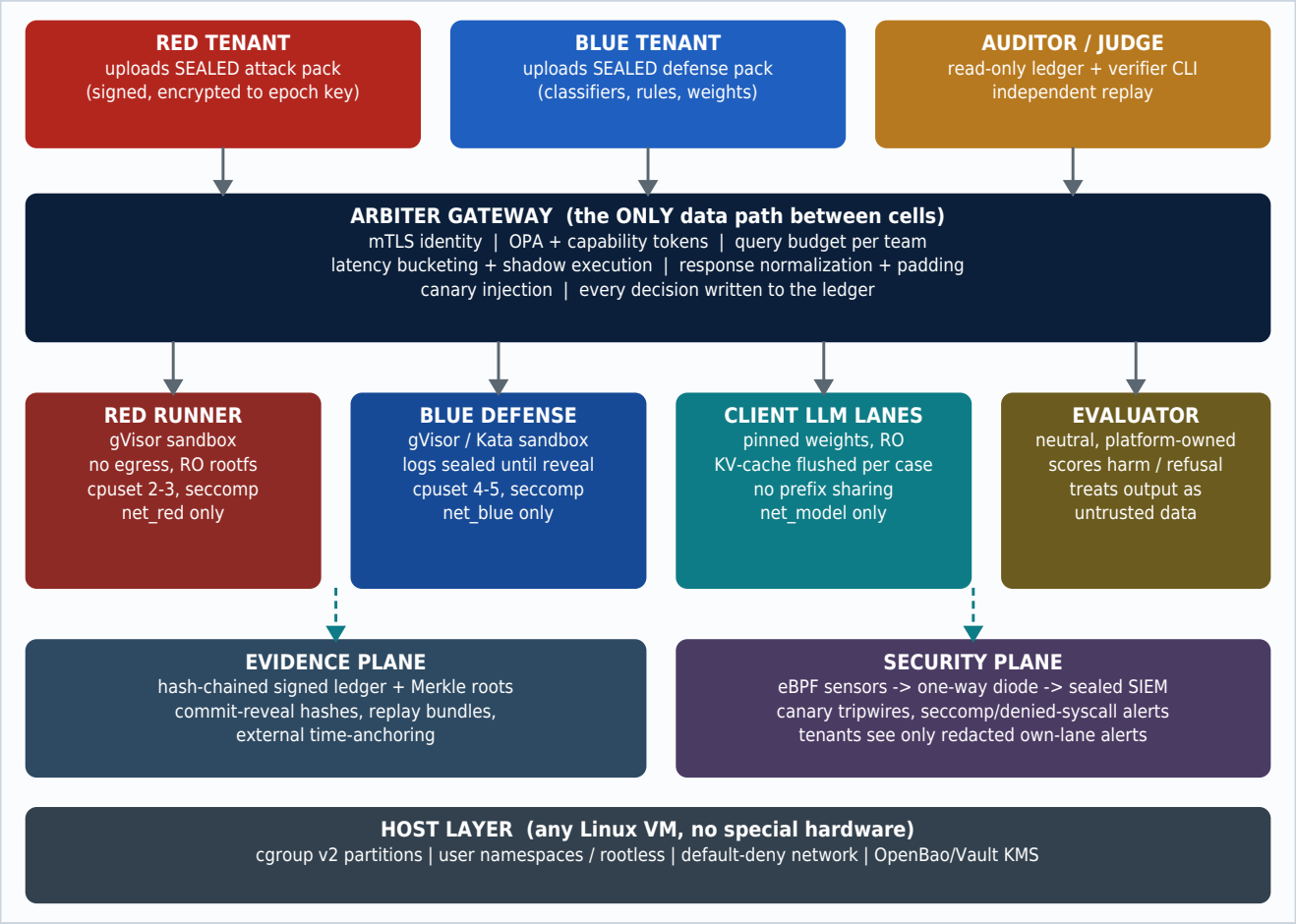
Hidden contradiction	Why naive isolation fails	Bastion Arena answer
Blue must inspect attacks to defend, yet must not observe red payloads before the test ends	If blue runs live defenses it sees payloads; if it cannot, there is no defense to test	**Blind execution:** blue uploads a *sealed defense pack* that runs in a platform-owned sandbox. Humans and tooling on the blue side never see inputs, logs or intermediate state until epoch reveal.
Red needs feedback to adapt, yet feedback leaks defensive logic	Block-vs-allow differences, timing and error text form an oracle that lets red reverse-engineer blue	**Controlled oracle:** normalized verdicts, constant-size responses, latency bucketing, *shadow execution* of the model on blocked inputs, and a hard per-team query budget that gives a provable leakage upper bound.
Isolation adds latency, yet evaluation must stay real-time and fair	Heavy sandboxes and noisy neighbours distort timing-based findings	**Governed fairness:** pinned cpusets, cgroup v2 quotas, identical resources per tenant, and a published latency-overhead budget measured by a benchmark in the repo.
Monitoring must detect leaks without becoming a leak	Shared dashboards and logs are the classic side channel	**Security plane with a one-way diode:** sensors write to a sealed SIEM; tenants only get redacted, quantized, own-lane signals. Canary tokens detect leaks by construction.

The three ideas that carry the project

- **Sealed packs + Arbiter:** tenants never share a runtime or a network. Everything crosses one audited, policy-enforcing choke point.
- **Commit-reveal epochs:** hashes of attack and defense packs are committed *before* a run and revealed *after*, so neither side can cheat and findings are bound to exactly what was tested.
- **Verifiable evidence:** a signed, hash-chained, Merkle-anchored ledger plus a replay bundle lets a third party re-verify any finding with one command, and the architecture ships with an honest, quantified residual-risk statement.

What makes it memorable to judges: the demo ends with *us attacking our own platform* (an automated Isolation Scorecard of 12 breakout, leakage and tampering attempts) and an independent auditor CLI that detects a tampered log. Security claims you can watch fail to be broken are worth more than slides.

2. System architecture



Planes. Control plane: Arbiter, epoch manager, policy engine (OPA), key broker. **Data plane:** the four sealed cells (Red Runner, Blue Defense, Client LLM lanes, Evaluator). **Evidence plane:** ledger, commit-reveal store, replay bundles. **Security plane:** sensors, canaries, sealed SIEM. Splitting planes means a compromise of one cell cannot rewrite evidence or policy.

2.1 Request path for one test case

- Red Runner emits a candidate prompt to the Arbiter over a unix socket / mTLS channel (it has no other route out).
- Arbiter checks identity, epoch phase and remaining query budget (OPA), injects a per-epoch canary where configured, and forwards to the **Blue Defense** sandbox for *input inspection*.
- **Shadow execution:** the Arbiter always runs the Client LLM lane, even if blue blocked the input. This removes the “blocked requests return faster” timing oracle. The model output is stored for the evaluator but is released to red only if blue allowed it.
- Blue Defense inspects the output; the Evaluator (neutral, platform-owned) scores harmfulness and policy compliance.
- Arbiter normalizes the verdict to a fixed schema, pads the response to a fixed size, waits to the next latency bucket, appends a hash-only event to the ledger, and returns the public view to red.

2.2 Who may see what

Artifact	Red	Blue	Client LLM	Auditor	Platform admin
Attack payloads (during epoch)	own pack only	never	sees the prompt, nothing else	hash only	encrypted, no key
Defense logic / weights (during epoch)	never	own pack only	never	hash only	encrypted, no key
Model weights / state	never	never	pinned, read-only	hash + attestation	operates, cannot alter silently
Per-case verdict stream	normalized only	never	never	full, after close	metadata only
After reveal	own outcomes	evaded attacks + own logs	n/a	everything	everything + audit trail

2.3 Epoch protocol (commit → seal → run → close → reveal → verify)

Phase	What happens	Ledger event
1. Commit	Each tenant uploads a pack; platform stores ciphertext and publishes SHA-256 commitments of red pack, blue pack, model image/weights, evaluator version, seeds, resource profile.	EPOCH_COMMIT{hashes}
2. Seal	Epoch key generated in Vault/OpenBao; packs decrypted only inside their sandboxes. Network policy flips to run mode. Model weights hash verified.	EPOCH_SEAL{model_hash, policy_hash}
3. Run	Test cases flow through the Arbiter. Canaries are live. Security plane is watching.	CASE{id, in_hash, out_hash, verdict}
4. Close	Budget exhausted or time elapsed. Sandboxes are destroyed, model hash re-verified, Merkle root signed and anchored externally.	EPOCH_CLOSE{merkle_root, model_hash_after}
5. Reveal	Policy-defined disclosure: red sees outcomes, blue sees evaded attacks, auditor sees everything. Keys released per policy.	REVEAL{who, what}
6. Verify	Auditor CLI recomputes the chain, checks signatures, replays the epoch from the bundle and compares verdict hashes.	VERIFY{result}

3. Deep dive on the seven technical verticals

Each vertical lists the threat, the control, the tool, and the **proof you will demo**. The “proof” column is what separates a working PoC from a paper design.

3.1 Container and sandbox isolation

Layer	Control	Tool
Execution boundary	User-space kernel so a tenant syscall never reaches the host kernel directly. Optional microVM tier.	gVisor (runsc); Kata Containers or Firecracker as stretch
Privilege	Rootless engine, user namespaces, *cap_drop ALL*, *no-new-privileges*, read-only rootfs, noexec tmpfs, pids limit.	Docker rootless / Podman, compose security_opt
Syscall surface	Per-role seccomp allow-list (tenant packs need very few syscalls), AppArmor profile, no ptrace, no mount, no raw sockets.	seccomp JSON, AppArmor
Filesystem	Packs mounted read-only; scratch is per-case tmpfs, wiped between cases; no shared volumes between cells.	tmpfs, overlay RO

Proof: Scorecard tests T1–T4 (mount escape, /proc peeking, ptrace, forbidden syscall) all fail and each raises a security-plane alert.

3.2 Network segmentation and traffic control

- One **internal** network per cell (*net_red, net_blue, net_model, net_eval*). Only the Arbiter is attached to more than one. There is no route between cells and no route to the internet.
- Default-deny at two layers: the engine network (iptables / Cilium NetworkPolicy) and the Arbiter's OPA policy (identity + phase + budget).
- Mutual TLS with short-lived workload identities (step-ca for the PoC; SPIFFE/SPIRE for production). DNS is disabled inside cells.
- Uniform message framing: fixed-size padded frames and a fixed verdict schema remove payload-length and error-text side channels.

Proof: T5–T7 attempt cell-to-cell TCP, DNS tunnelling and ICMP from the red cell; all drop, all alert.

3.3 Access control and secrets management

- **Capability tokens** (Biscuit or macaroons) scoped by team, epoch, action and expiry, e.g. *red:attack.submit@epoch-17*. Attenuation lets the Arbiter hand a runner exactly the capabilities it needs and no more.
- **ABAC via OPA/Rego** on role, epoch phase, resource owner and sensitivity label. Phase is an attribute, so results are unreadable until *revealed* by policy, not by convention.
- **Envelope encryption:** each pack is encrypted to a per-epoch data key held in OpenBao/Vault (transit engine). Platform admins can store ciphertext but cannot decrypt without a two-person key release.
- **Secrets hygiene:** no secrets in images or env vars; tokens injected via tmpfs, TTL under one epoch; automatic rotation at close.

```
# policy/authz.rego  (OPA)
package bastion.authz
import rego.v1
default allow := false

allow if { input.role == "red";  input.action == "attack.submit";  input.epoch.phase == "commit" }
allow if { input.role == "blue"; input.action == "defense.submit"; input.epoch.phase == "commit" }
allow if { input.role == "red";  input.action == "case.send"; input.epoch.phase == "run"
           input.budget.remaining > 0 }
allow if { input.action == "results.read"; input.resource.owner == input.team
           input.epoch.phase == "revealed" }
allow if { input.role == "auditor"; input.action == "ledger.read" }
```

3.4 Audit trails and forensic provenance

- **Hash chain + signatures:** every event is $h_i = SHA256(h_{i-1} \,||\, canonical(event))$, signed with an Ed25519 key held in the key broker. Payloads are stored as hashes, so the ledger itself leaks nothing.
- **Merkle tree per epoch:** gives O(log n) inclusion proofs (“prove case 481 was in this epoch”) and a single root to anchor.

- **External anchoring:** publish the epoch root to an RFC 3161 timestamp authority or a public transparency log (Sigstore Rekor), so even the platform operator cannot silently rewrite history.
- **Artifact provenance:** cosign signatures on images and packs, SBOMs (Syft), in-toto attestations for the build; digests pinned in the epoch manifest.
- **Replay bundle:** manifest of image digests, model hash, seeds, policies, resource profile and kernel version. Run *bastion verify --replay epoch-17* to re-execute at temperature 0 and compare verdict hashes.

```
import hashlib, json
from nacl.signing import SigningKey          # pip install pynacl

class Ledger:
    def __init__(self, sk: SigningKey):
        self.sk, self.prev, self.n = sk, b"\x00" * 32, 0
    def append(self, event: dict) -> dict:
        body = json.dumps(event, sort_keys=True, separators=(",", ":")).encode()
        h = hashlib.sha256(self.prev + body).digest()
        rec = {"n": self.n, "prev": self.prev.hex(), "h": h.hex(),
               "sig": self.sk.sign(h).signature.hex(), "event": event}
        self.prev, self.n = h, self.n + 1
        return rec

def verify(records, vk):                     # auditor side: any edit/reorder/delete fails
    prev = b"\x00" * 32
    for r in records:
        body = json.dumps(r["event"], sort_keys=True, separators=(",", ":")).encode()
        h = hashlib.sha256(prev + body).digest()
        assert h.hex() == r["h"] and r["prev"] == prev.hex()
        vk.verify(h, bytes.fromhex(r["sig"]))
        prev = h
    return True
```

Proof: live demo flips one byte in the stored ledger; the auditor CLI pinpoints the first corrupted record.

3.5 Resource governance and fairness

- **cgroup v2 partitions** per cell: *cpuset.cpus* pinning (disjoint cores), *memory.max*, *io.max*, *pids.max*. Identical profiles for red and blue so neither is advantaged.
- **Timing hygiene:** latency bucketing in the Arbiter (responses released on a fixed grid), fixed-size frames, disabled SMT on the host where possible, and no tenant-visible high-resolution timers beyond what the API exposes.
- **Noisy-neighbour test:** red burns CPU/IO; the evaluation's latency distribution and verdicts must not shift beyond a pre-declared tolerance (e.g. p95 latency drift under 5 percent, verdict flips = 0).
- **Model lanes:** N identical workers with fair scheduling; each test case is bound to a lane and the lane's KV cache is flushed afterwards.

3.6 LLM-specific threat surface

Threat	How it works	Control in Bastion Arena
Shared KV / prefix-cache side channel	A tenant infers another tenant's prompt prefix from time-to-first-token differences when a cache is shared (published research has demonstrated this on multi-tenant serving).	No cross-tenant prefix sharing: disable prefix caching or use per-tenant cache salts where the server supports it; flush slots per case. **Verified by a statistical probe** (AUC about 0.5).
Prompt-context leakage	Earlier test cases bleed into later ones through retained context or conversation state.	Stateless lanes; fresh context per case; no memory features; context hash logged.
Cross-session contamination	Model state or weights altered by one session affect another.	Read-only pinned weights; SHA-256 verified before seal and after close; no online learning or adapters.
Output-side exfiltration	A compromised model or pack hides secrets in outputs (encoding, steganography, long tails).	Egress only via Arbiter; output size caps; canary detection in all outputs and logs; structured parse in Evaluator.
Evaluator injection	Model output contains instructions that manipulate the judge.	Evaluator has no tools or network; model output is wrapped as data; rule-based checks run first; judge prompt is fixed and hashed.
Logit / logprob leakage	Token probabilities reveal defense or model details.	Logprobs never exposed to tenants; decoding parameters fixed by the manifest.

```
# Statistical KV-cache probe (runs in the Scorecard, prefix cache ON vs OFF)
prime(tenant="A", prefix=P)                                  # A uses prefix P
t_hit  = [ttft(tenant="B", prompt=P + rand_suffix()) for _ in range(300)]
t_miss = [ttft(tenant="B", prompt=Q + rand_suffix()) for _ in range(300)]
auc = roc_auc_score([1]*300 + [0]*300, [-t for t in t_hit + t_miss])
# cache ON  -> auc >> 0.5  (leak visible, demo of the vulnerability)
# cache OFF / salted -> auc ~ 0.50 +/- 0.03  (leak closed)
```

3.7 Observability and anomaly detection (without a leak)

- **One-way diode:** sensors (Tetragon or Falco eBPF, auditd fallback) emit to a sealed SIEM; no tenant can query it. The SIEM never feeds raw content back to cells.
- **Tenant-visible telemetry is deliberately poor:** own-lane only, aggregated, quantized to coarse buckets, delayed to fixed intervals; alerts say “policy violation in your lane” with no cross-tenant identifiers.
- **Detections:** denied syscalls, unexpected socket/DNS attempts, reads of foreign paths, cgroup pressure correlated with timing anomalies, ledger gaps, model-hash drift, **canary hits** (a canary seen outside its lane is proof of a leak).
- **Canaries:** unique per-epoch tokens embedded in red payload sets and blue rules; any sighting elsewhere (inputs, outputs, logs, metrics) trips an alarm.

4. Threat model and residual risk

Assets: red payloads, blue defenses and weights, client model state, evaluation verdicts, the evidence ledger, signing and epoch keys. **Adversaries:** (A1) malicious red tenant, (A2) curious blue tenant, (A3) compromised or adversarial client model, (A4) external network attacker, (A5) honest-but-curious platform operator, (A6) malicious supply-chain component.

Risk	Mitigation	Residual risk and when guarantees fail	Future fix
Sandbox escape via kernel or runtime 0-day	gVisor, seccomp, rootless, cap drop, patch cadence	A bug in runsc or the host kernel defeats isolation. Fails if the host is compromised.	Firecracker microVM per tenant; dedicated nodes
Verdict oracle: red infers defense from outputs	Normalized verdicts, shadow execution, query budget	Cannot reach zero: red must observe something. Information is bounded by about Q x log2(|V|) bits for Q queries and verdict set V.	Differential-privacy noise on verdicts; adaptive budgets
Timing channels (Arbiter, model, queueing)	Bucketing, padding, fixed frames, cpuset	Coarse timing leaks remain if a bucket is exceeded or under heavy load.	Constant-rate scheduling; cover traffic
Microarchitectural (shared caches, Spectre-class)	Disjoint cores, SMT off, no shared memory between cells	Remains possible on shared physical hosts and shared GPUs.	Per-tenant nodes; confidential VMs (SEV-SNP, TDX); GPU MIG
Platform operator abuse	Envelope encryption, two-person key release, ledger records admin actions	A root operator can still tamper with the running system; the ledger shows evidence but does not prevent it.	Remote attestation + confidential computing
Ledger tampering or truncation	Hash chain, signatures, external anchoring	Key compromise or truncation before the next anchor window.	HSM keys; witness cosigning; shorter anchor interval
Supply chain (packs, images, model weights)	cosign, SBOM, pinned digests, no network at build	Unknown malicious dependency inside a tenant pack runs inside the sandbox.	Reproducible builds; pack static analysis
Malicious client model	Egress only via Arbiter, evaluator hardening, output caps	Subtle encoding of data in permitted outputs within size caps.	Output entropy checks; paraphrasing gate
Monitoring as a leak	Diode, redaction, quantization	Metadata about alert timing can still carry bits at low rates.	Fixed-interval batched alerts
Nondeterministic replay (GPU kernels)	Temp 0, fixed seeds, CPU deterministic mode for evidence runs	Bit-exact replay not guaranteed on all GPUs.	Deterministic inference kernels

What the architecture protects against: cross-tenant data access, network lateral movement, direct payload or defense disclosure, shared-cache prompt leakage, silent log tampering, resource-based result distortion. **What it does not protect against:** a fully compromised host or hypervisor, a malicious cloud provider, physical attacks, and the information-theoretic minimum leakage through the verdict channel. Stating this clearly is itself a

scoring objective (Objective 06) and builds judge trust.

5. Technology stack

Concern	PoC tier (works on a single Linux VM)	Production tier (stretch)
Runtime isolation	Docker/Podman rootless + gVisor (runsc), seccomp, AppArmor	Kata / Firecracker microVMs, Kubernetes RuntimeClass
Orchestration	Docker Compose + Makefile	k3s/Kubernetes + custom Operator (CRDs: Epoch, Pack)
Network	Internal Docker networks, iptables default-deny, step-ca mTLS	Cilium (eBPF, L7 policy), SPIFFE/SPIRE, Envoy
Policy / access	OPA (Rego), Biscuit tokens	OpenFGA relationship model + OPA, attribute-based labels
Secrets / KMS	OpenBao or Vault dev server (transit), PyNaCl / age	HSM or cloud KMS, Shamir-split unseal
Audit / provenance	Python Merkle + hash-chain ledger (SQLite/Postgres), cosign, Syft, in-toto	Sigstore Rekor / Trillian-class log, witness cosigning, RFC 3161 TSA
LLM serving	llama.cpp server or Ollama with a small GGUF model (Qwen or Llama 1–3B class); vLLM if a GPU is available	vLLM with per-tenant cache salts, dedicated GPU or MIG partitions
Blue defenses (sample)	Regex/keyword baseline, embedding + scikit-learn classifier, small guard model	Customer classifiers, Llama-Guard-class models, ensembles
Evaluator	Refusal-pattern matcher + small LLM judge in a sandbox	Calibrated classifiers (e.g. HarmBench-style), human review queue
Red sample corpora	JailbreakBench, HarmBench, AdvBench, Gandalf prompts (small samples)	Adaptive attack generators (e.g. garak, PyRIT adapters as packs)
Observability	Falco or Tetragon on the VM, auditd fallback, Prometheus + Grafana (security plane only)	OpenTelemetry, sealed SIEM, anomaly models
Services / UI	Python 3.12, FastAPI, gRPC, pydantic; Streamlit or React dashboard (Merkle viewer)	Go/Rust data path, Next.js console
Testing / CI	pytest, hypothesis, SciPy (stat tests), k6 (load), GitHub Actions	Fuzzing the Arbiter, formal model (TLA+)
Infra	Ubuntu 24.04 VM with cgroup v2 (any cloud, no special hardware)	Terraform, multi-node, optional confidential VMs

Practical tip: gVisor needs a real Linux kernel. Use a cloud VM or a local Linux VM, not Docker Desktop on macOS/Windows. If runsc is unavailable at demo time, the scorecard should auto-report a *degraded mode* (runc + seccomp + user namespaces) rather than silently claiming full isolation.

6. Implementation playbook

6.1 Repository layout

```
bastion-arena/
  arbiter/            FastAPI + gRPC gateway: authz, budget, bucketing, shadow exec, normalization
  cells/
    red-runner/       sandbox harness that loads a sealed attack pack (unix socket only)
    blue-defense/     sandbox harness that loads a sealed defense pack (check_input/check_output)
    model-lane/       llama.cpp / vLLM wrapper: pinned weights, KV flush, no logprobs
    evaluator/        neutral judge, output treated as untrusted data
  ledger/             hash chain + Merkle + signing + anchoring + verify CLI
  policy/             authz.rego, network policy, seccomp/*.json, apparmor/*
  keys/               OpenBao bootstrap, epoch key broker
  secmon/             Tetragon/Falco rules, canary manager, SIEM sink (diode)
  scorecard/          12 automated attack tests + KV-cache probe + noisy-neighbour test
  packs/              sample red packs (JailbreakBench/HarmBench slices), sample blue packs
  dashboard/          Merkle explorer, epoch timeline, scorecard view
  docs/               threat-model.md, architecture.md, pitch deck
  Makefile            make demo | make scorecard | make verify EPOCH=17
```

6.2 Docker Compose hardening pattern

```
networks:
  net_red:   { internal: true }
  net_blue:  { internal: true }
  net_model: { internal: true }

services:
  arbiter:                      # only service attached to multiple networks
    networks: [net_red, net_blue, net_model]
  red-runner:
    runtime: runsc              # gVisor
    networks: [net_red]
    read_only: true
    cap_drop: [ALL]
    security_opt: ["no-new-privileges:true", "seccomp=./policy/seccomp/tenant.json"]
    cpuset: "2-3"
    mem_limit: 2g
    pids_limit: 256
    tmpfs: ["/tmp:size=64m,noexec,nosuid"]
    dns: []
  blue-defense:                 # identical profile on cores 4-5, net_blue only
    runtime: runsc
    networks: [net_blue]
    cpuset: "4-5"
```

6.3 The Arbiter hot path (timing-safe)

```
async def handle_case(case, team):
    t0 = loop.time()
    authz.require(team, "case.send", epoch)                 # OPA + budget
    case = canary.inject(case, epoch)
    g_in  = await blue.check_input(case.prompt)             # sealed sandbox
    out   = await model.generate(case.prompt, lane=pick())  # ALWAYS run (shadow exec)
    g_out = await blue.check_output(out)
    score = await evaluator.judge(case.prompt, out)         # neutral, platform-owned
    verdict = normalize(g_in, g_out)                        # {PASS | BLOCK} only
    public = pad(release(out) if verdict == "PASS" else REFUSAL_STUB)
    await sleep_until(t0 + bucket(loop.time() - t0))        # latency quantization
    ledger.append({"case": case.id, "in": h(case.prompt), "out": h(out),
                   "verdict": verdict, "score": score, "lane": lane_id})
    return public
```

6.4 Build order (48-hour plan; compress proportionally for 24 hours)

Window	Deliverable	Done when
H0–4	Repo, VM with gVisor, compose skeleton with four internal networks, model lane serving a small model, mock red/blue packs	Red prompt reaches model only via Arbiter
H4–12	Arbiter with mTLS, OPA authz, budget, normalization, bucketing, shadow execution	Block vs allow latencies statistically indistinguishable
H12–20	Ledger (chain + Merkle + signatures), verify CLI, commit-reveal epoch manager, OpenBao envelope encryption	Tamper test fails verification
H20–28	Hardening: seccomp, cap drop, cpuset/cgroups, security plane sensors, canaries, redacted tenant telemetry	Scorecard T1–T9 pass
H28–36	LLM threat surface: KV-cache probe on/off, stateless lanes, weight attestation, evaluator injection test	AUC about 0.5 with mitigation

Window	Deliverable	Done when
H36–42	Dashboard (Merkle explorer, epoch timeline, scorecard), replay bundle, one-command *make demo*	Clean-VM install under 10 minutes
H42–48	Threat model doc, benchmarks (latency overhead, noisy neighbour), pitch rehearsal, backup video	Demo runs twice without edits

6.5 Priority tiers (so you never ship a half-working system)

Tier	Items
Must (core score)	Per-cell sandboxing and internal networks; Arbiter as sole path; commit-reveal; hash-chained signed ledger + verifier; seccomp/cgroups; KV-cache isolation with probe; threat model with residual risk.
Should (separates winners)	Shadow execution and latency bucketing with a measured result; canaries; OPA capability model; envelope encryption; Isolation Scorecard; replay bundle.
Could (wow factor)	Merkle explorer UI; external timestamp anchoring; Kata/Firecracker tier; Cilium policies; second model lane pool; k6 load report.

If you are solo or short on time, ship the Must tier flawlessly and demo one Should item deeply. A smaller system that cannot be broken in front of the judges beats a larger one that can.

7. Validation: the Isolation Scorecard

Run automatically (*make scorecard*) and show the output live. Each test is an attack *we* launch against Bastion Arena from inside a tenant cell.

ID	Attack attempted	Expected result	Evidence shown
T1	Red reads blue pack path / mounts host filesystem	Permission denied; no mount	Seccomp audit alert
T2	Red enumerates /proc and cgroup files of other cells	Not visible	Sandbox namespace proof
T3	ptrace / raw socket / unshare in tenant cell	Syscall blocked	Seccomp kill + alert
T4	Write to root filesystem, drop persistence	Read-only filesystem	Failed write log
T5	Red opens TCP to blue/model addresses	No route	Network drop counter
T6	DNS tunnelling and ICMP covert channel to outside	No resolver, no egress	Dropped packets + alert
T7	Red tries to call model lane directly	Unreachable; only Arbiter works	Policy deny event
T8	Canary from red set appears in blue logs/metrics?	Never; alarm if yes	Canary scan report
T9	Block vs allow timing distinguishable by red?	AUC about 0.5	ROC plot before/after
T10	KV-cache prefix probe across tenants	AUC about 0.5 with mitigation	ROC plot cache ON vs OFF
T11	Noisy neighbour: red burns CPU/IO	p95 drift within tolerance; zero verdict flips	Latency histograms
T12	Tamper with ledger / delete a record / reorder	Verifier fails at first bad record	Auditor CLI output

7.1 Metrics to publish on one slide

- **Isolation:** 12/12 breakout and leakage attempts blocked; mean time to alert for each.
- **Leakage:** timing-oracle AUC and KV-probe AUC with mitigation (target 0.50 ± 0.03) versus unmitigated baseline.
- **Overhead:** p50/p95 latency of direct model call versus Arbiter path, and cost of shadow execution (extra model compute on blocked cases).
- **Integrity:** time to verify an epoch of N cases; tamper detection rate 100 percent in test.
- **Fairness:** verdict stability and latency drift under noisy-neighbour load.
- **Deployability:** time from clean Ubuntu VM to *make demo* (target under 10 minutes).

Targets above are goals to measure and report honestly; do not present them as results until you have run the benchmarks.

7.2 Mapping to the six evaluation objectives

Objective	What we deliver	Proof in the demo
01 Isolate without breaking function	Sealed cells, single Arbiter path, cpuset/cgroup governance, latency budget	Overhead benchmark, live epoch

Objective	What we deliver	Proof in the demo
02 Prevent information leakage	Blind execution, normalization, shadow exec, canaries, redacted telemetry	T8, T9, leakage-bound statement
03 Guarantee audit integrity	Hash chain, signatures, Merkle, anchoring, replay bundle	T12, auditor replay
04 Build for deployability	Standard Linux VM, Compose, open-source tools only, no special hardware	make demo on a clean VM
05 Address LLM-native threats	KV-cache isolation, stateless lanes, weight attestation, evaluator hardening	T10, hash attestation
06 Quantify residual risk	Threat model table with failure conditions and roadmap	Section 4 on a slide

7.3 Five-minute demo script

- **0:00 Hook:** “If the lab leaks, the finding is worthless.” Show the architecture diagram, one sentence per plane.
- **0:45 Normal epoch:** commit hashes, seal, run 100 cases from JailbreakBench/HarmBench slices; dashboard shows verdicts and ledger growth.
- **1:45 Attack the platform:** run the scorecard; show T1–T7 blocked and the security-plane alerts firing.
- **2:45 Side channels:** show KV-cache ROC curve with cache ON (leak) versus mitigated (flat), and the timing-oracle plot.
- **3:45 Integrity:** flip one byte in the ledger; auditor CLI names the corrupted record; then replay the epoch successfully.
- **4:30 Honesty slide:** residual risk table and roadmap to confidential computing. Finish on the tagline.

8. How to present it so judges score you high

- **Lead with the contradictions** (Section 1). It signals depth: you solved the real problem, not the easy one.
- **Show failures, not just success:** an attack that is attempted and visibly blocked is stronger than any diagram.
- **Be quantitative:** AUC values, p95 overhead, bits-of-leakage bound. Numbers beat adjectives.
- **Be honest about limits:** a clear residual-risk slide is explicitly an evaluation objective; overclaiming loses credibility with security judges.
- **Use standard tools:** judges reward reproducibility. Everything open source, one command to run, no proprietary hardware.
- **Have a backup:** record a clean demo video and keep a pre-built VM snapshot in case venue Wi-Fi or hardware fails.
- **Expect these questions:** “What if the Arbiter is compromised?” (single trusted component, small, auditable, fuzzed; next step attestation). “Can red still infer something?” (yes, bounded by query budget; here is the bound). “Why gVisor, not just Docker?” (user-space kernel shrinks host syscall surface). “How do I know the ledger is not rewritten by you?” (external anchoring and independent verifier).

Datasets and how to use them

Resource	Role in the PoC
JailbreakBench, HarmBench, AdvBench	Red pack content: small curated slices of attack prompts; HarmBench-style categories for evaluator scoring.
Gandalf / Lakera dataset	Prompt-injection and instruction-override samples for the injection track and evaluator-injection test.
ToxiGen, HH-RLHF	Optional: toxicity and harmlessness probes, and sanity data for blue classifiers.
NVD / CVE	Threat-model grounding and patch-cadence argument for runtime and kernel risks (cite relevant runc/gVisor/kernel escapes).
DARPA Transparent Computing	Inspiration for provenance graph design in the ledger and replay.

Use these datasets only as test inputs inside the sandboxes; keep any harmful content out of slides and screenshots.

9. Future enhancements

Horizon	Enhancement	Why it matters
0–6 months	Kubernetes Operator (Epoch, Pack CRDs), Cilium L7 policy, SPIFFE/SPIRE identities, Kata/Firecracker tier	Moves from PoC to multi-node, multi-tenant service

Horizon	Enhancement	Why it matters
0–6 months	Pack SDK and adapters for garak, PyRIT, Promptfoo, Inspect; compliance report generator (EU AI Act, NIST AI RMF, ISO/IEC 42001)	Makes the platform the trust layer under existing tools and speaks the auditor's language
6–12 months	Confidential computing with remote attestation (AMD SEV-SNP, Intel TDX, NVIDIA confidential GPUs)	Removes the cloud operator from the trusted base; attested epochs
6–12 months	Differential-privacy noise on the verdict channel and adaptive query budgets	Turns the leakage bound into a formal privacy guarantee
6–12 months	Per-tenant GPUs / MIG, deterministic inference kernels, bit-exact replay	Closes GPU side channels and strengthens reproducibility
12–24 months	Formal verification of the epoch protocol (TLA+/Alloy) and Arbiter in a memory-safe language with fuzzing	Strengthens guarantees for regulated customers
12–24 months	Agentic and tool-use red teaming sandbox (simulated tools, egress honeypots), multimodal models	Matches where real safety risk is moving
12–24 months	Federated mode: cross-organisation evaluations with cryptographic evidence exchange; open Evidence Bundle standard	Enables third-party audits and shared safety benchmarks without sharing secrets

10. Market value and economics

10.1 Market size (third-party estimates, directional)

Published analyst estimates for AI red-teaming and AI security testing differ widely because segments and methods differ, but all point to fast growth driven by regulation and enterprise LLM adoption:

Source (analyst report)	Estimate	Notes
Technavio, AI Red Teaming Solutions Market 2026–2030	Growth of about USD 6.69 billion over 2025–2030, CAGR about 25.1 percent	Software platforms cited as the central segment; regulatory mandates cited as primary driver
Astute Analytica, AI Red Teaming and Security Testing Market	About USD 0.5 billion (2025) to about USD 8.0 billion (2035), CAGR about 32 percent	Segments: platforms/tools and managed red-teaming services; BFSI, healthcare, government and defense, IT

These are vendor-published forecasts; quote them as estimates and name the source on the slide.

10.2 Why now (demand drivers)

- **Regulation:** EU AI Act obligations for general-purpose and high-risk AI, NIST AI Risk Management Framework, and ISO/IEC 42001 push organisations toward documented adversarial testing with evidence.
- **Institutional demand:** national AI safety and security institutes and frontier labs run structured evaluations and need defensible, reproducible results.
- **Enterprise rollout:** banks, healthcare and government deploy LLMs and must show pre-deployment assurance to auditors and insurers.
- **Trust gap:** today most red-team results are screenshots and spreadsheets. Evidence-grade, tamper-evident results are a missing layer.
- **Consolidation signal:** AI-security startups have been acquired by large security vendors, which indicates strategic value in the category.

10.3 Customers and positioning

Segment	Pain	Value of Bastion Arena
AI labs and model providers	Run internal red/blue programmes; leaks invalidate results	Isolated, repeatable, attestable evaluation environment
Regulated enterprises (BFSI, health, public sector)	Must evidence pre-deployment testing to auditors	Audit-ready evidence bundles and compliance reports
Third-party evaluators and audit firms	Need credibility and independence	Neutral arena where both sides cannot cheat; verifiable output
Cloud and MLOps platforms	Want AI safety as a managed service	White-label tenant-isolated testing layer
AI safety institutes and benchmark groups	Need shared benchmarks without leaking test sets	Sealed test sets with commit-reveal and replay

Positioning: open-source red-team tools (for example garak, PyRIT, Promptfoo, HarmBench, Inspect) generate and score attacks; security vendors sell guardrails and scanners. None of these is mainly about *trustworthy, isolated, verifiable execution* of a live red/blue contest. Bastion Arena is the trust layer that hosts those tools as packs.

10.4 Business model

- **Platform subscription** (per organisation, tiered by concurrent epochs and retained evidence).
- **Usage-based compute:** per evaluation-hour and per million test cases (includes shadow-execution compute).
- **Assurance as a service:** signed evidence bundles, auditor access seats, managed epochs run by the vendor.
- **Regulated on-prem / VPC licence** with support SLA and attested-hardware add-on.
- **Open-core:** open Evidence Bundle spec and verifier (drives trust and adoption); paid control plane, compliance packs and enterprise features.

10.5 Illustrative unit economics (assumptions, not forecasts)

Year	Paying customers	Average annual contract	Annual recurring revenue
Year 1	10	USD 40,000	USD 0.4 million
Year 2	40	USD 55,000	USD 2.2 million
Year 3	120	USD 70,000	USD 8.4 million

These figures are a pitch illustration built from simple assumptions (customer count and contract size). Replace them with numbers from your own customer-discovery calls before presenting them as a business plan. Year-3 revenue under this assumption would be roughly the order of 0.1 percent of the larger analyst estimate for 2035 and a small share of the 2030 estimate, which is why we call it conservative.

10.6 Go-to-market

- **Phase 1:** open-source the verifier and scorecard; publish the threat model; win the hackathon visibility; pilot with a university lab or AI safety group.
- **Phase 2:** design partners among regulated enterprises and AI audit firms; integrate with garak/PyRIT/Inspect packs; compliance report templates.
- **Phase 3:** cloud-marketplace listing and partnerships with MLOps and cloud providers; attested-hardware tier for government and defence.
- **Moat:** evidence format adoption, integrations, accumulated attestations, and security credibility (independent audits, bug bounty on the Arbiter).

10.7 Risks to the business and answers

Risk	Answer
Cloud providers add similar features	Neutral, multi-cloud evidence standard and cross-vendor verifier; partner rather than compete
Guarantees overclaimed	Published residual-risk model; independent security review; attestation roadmap
Customers see it as niche	Position as compliance infrastructure for every LLM deployment, not only frontier labs
Open-source tools bundle isolation	Be the maintained integration layer and the evidence authority

Appendix A. Quick-start commands (reference)

```
# 1. Host prep (Ubuntu 24.04, cgroup v2)
sudo apt install -y docker.io                          # or rootless Docker / Podman
# install gVisor (runsc) per the official docs, then register the runtime:
sudo runsc install && sudo systemctl restart docker

# 2. Bring up the arena
make keys        # OpenBao dev bootstrap + epoch key broker
make demo        # compose up: arbiter, cells, ledger, secmon, dashboard
make scorecard   # run T1-T12 attack battery + KV/timing probes

# 3. Independent verification (auditor side)
bastion verify --epoch 17 --pubkey auditor.pub          # chain + signatures + Merkle
bastion verify --epoch 17 --replay                      # deterministic replay vs committed hashes
```

Final advice: build less, but make every claim demonstrable. Judges remember a system that tried to break itself in public and published exactly where it would still fail.